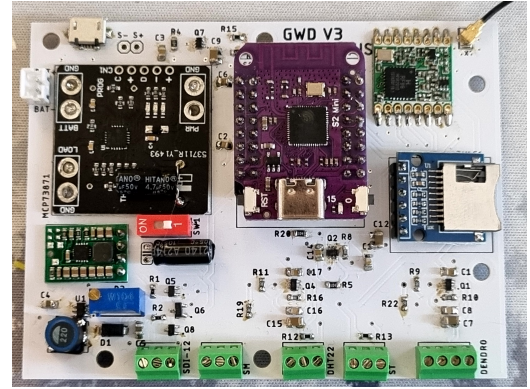


WiLo V1.0 (Wireless logger)



Overview

- I2C, SDI-12, One-Wire communication protocols
- 40 day battery life when connected to 3.7 V 5000 mAh Li-Po battery
- Environmental Sensing
- LoRa network integration
- Local SD storage
- Solar powered

Description

- The device is intended for field deployment, to measure various environmental metrics and/or use in precision forestry applications where tree growth and water dynamics must be measured.
- An SDI-12 terminal allows integration with SDI-12 sensors.
- One-Wire sensors (such as DS18B20 temperature sensor or DHT22 Temperature and humidity sensor) have dedicated terminals.
- I2C and UART capable.
- Battery voltage detection allows user to measure current battery voltage level.

Contents

1	Technical specification	3
2	Sensor Pinout	3
3	ESP32-S2 mini and Firmware Pinout	3
4	System Processes	5
4.1	Initial Startup	5
4.1.1	Set CPU frequency to 40 MHz	5
4.1.2	Activate WiFi capabilities for OTA (Over-the-Air) update	5
4.1.3	Initialise SD Card	5
4.1.4	Read/Write files from/to SD card	6
4.1.5	Check for dendrometer	6
4.1.6	Setup Complete	6
4.2	Measurements	6
4.2.1	Battery	6
4.2.2	Air Temperature and Humidity (DHT22)	6
4.2.3	Soil Temperature (DS18B20)	6
4.2.4	Soil Moisture (Capacitive Soil Moisture V1.2) / Apogee Pyronometer	6
4.2.5	Measure Dendrometer	6
4.2.6	Measure Sapflow	7
4.2.7	SDI-12	7
4.3	Write Data to SD Card	7
4.4	LoRa Setup	8
4.5	Wake-up Procedures	8
5	SDI-12 Installation and Configuration	8
5.1	SDI-12 Memory Addressing	8
5.2	SDI-12 Voltage Configuration	9
6	SD Card	10
6.1	Formatting SD card	10
6.2	SD Card Measurement Files	10
7	Firmware Upgrade	11
7.1	Configuring PlatformIO and Source Files in VS Code	11
7.2	Physical Connection Firmware Upgrade	12
7.3	Wireless Connection (OTA) Firmware Upgrade	12
7.3.1	Retrieving firmware.bin File	12
7.3.2	Setting up mobile device/PC for OTA	13
8	Debugging	14
8.1	Hardware Debugging	14
8.1.1	Battery	14
8.1.2	SD Card Errors	15
8.2	Sensor Errors	15

1 Technical specification

	Unit	Value
Battery Voltage	V	3.7 V
Rail Voltage	V	3.3 V
SDI-12 Boost Voltage	V	5–12 V
Maximum Active Current Draw	mA	400
Maximum Sleep Current Draw	μ A	500
Dimensions	mm $\times$ mm $\times$ mm	72x95x40

2 Sensor Pinout

Careful consideration is required when connecting sensors to the WiLo terminals (Fig. 1). The corresponding signals of each terminal pin is shown in Table

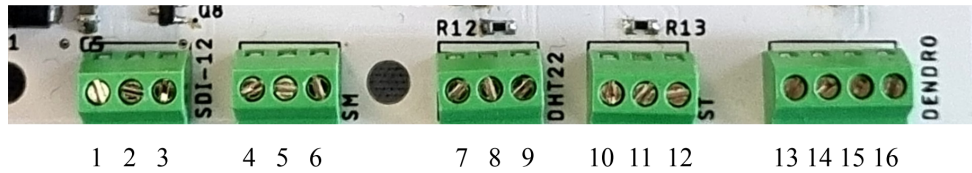


Figure 1: WiLo terminals. Each terminal has a name, indicating the sensor type connected. SDI-12: SDI-12 sensor. SM: Capacitive Soil moisture Sensor V1.2. DHT22: DHT22 Temperature and Humidity sensor. ST: 18B20 Soil Temperature Sensor. Dendro: Dendrometer, but these ports are also used for the sapflow sensor.

Table 1: Pin number and signal corresponding to WiLo terminals in Fig. 1.

Pin	Signal	Sensor Wire colour
1	Power 5–12 V, heat pulse enable	RD (SDI-12), WH (SF)
2	DATA SDI-12	WH
3	GND	BK
4	GND	BK/CLR
5	SM/PYRA DATA	YL/WH
6	Power 3.3 V	RD
7	Power 3.3 V	RD
8	DATA DHT22	YL/WH
9	GND	B
10	Power 3.3 V	RD
11	DATA ST	YL/WH
12	GND	BK
13	I2C SCL	YL
14	Power 3.3 V	RD
15	GND	BK
16	I2C SDA	BL

3 ESP32-S2 mini and Firmware Pinout

Table 2 describes the pinmapping to MCU pins of the ESP32-S2 mini microcontroller. These pins are defined within the Firmware of the WiLo. The pinmapping can be found in the `pinmapping.h`

header file of the WiLo source code, in the *include* folder.

Table 2: ESP32-S2 Mini Pin Mapping for WiLo (Wireless Logger)

Pin Name	Description	ESP32-S2 Mini Pin
Environmental Sensors		
DHTPIN	Data pin for DHT22 sensor	GPIO8
SOIL_MOISTURE_PIN	Analog signal from capacitive soil moisture sensor	GPIO9 (ADC1_CH8)
DHT22_SM_ENABLE_PIN	Enables power to DHT22 and soil moisture sensor	GPIO11
DS1B20_PIN	Data pin for DS18B20 temperature sensor	GPIO6
Battery Sensing		
BATTERY_ENABLE_PIN	Enables voltage divider for battery sensing	GPIO3
BATTERY_ADC_PIN	Reads battery voltage via ADC	GPIO1 (ADC1_CH0)
Sapflow & Dendrometer		
HEAT_PIN_SWITCH	Enables sapflow heat pulse (Marshall HPV)	GPIO11
SF_DENDRO_EN_PIN	Powers dendrometer and sapflow sensors	GPIO2
I2C_SDA	I2C data line	GPIO17
I2C_SCL	I2C clock line	GPIO21
SDI-12 Sensors		
SDI12_DATA_PIN	Data pin for SDI-12 sensors	GPIO5
SDI12_EN_PIN	Powers SDI-12 sensors	GPIO7
LoRa RFM95W (SPI1)		
LORA_SCK_PIN	SPI clock line for LoRa	GPIO36
LORA_MISO_PIN	SPI MISO for LoRa	GPIO37
LORA_MOSI_PIN	SPI MOSI for LoRa	GPIO35
LORA_CS_PIN	Chip select for LoRa module	GPIO34
LORA_RST_PIN	Reset pin for LoRa module	GPIO18
LORA_DIO0_PIN	Interrupt pin 0 for LoRa	GPIO33
LORA_DIO1_PIN	Interrupt pin 1 for LoRa	GPIO38
LORA_SPI	SPI bus number for LoRa module	SPI1
SD Card Interface (SPI2)		
SD_SCK_PIN	SPI clock line for SD card	GPIO36
SD_MISO_PIN	SPI MISO for SD card	GPIO37
SD_MOSI_PIN	SPI MOSI for SD card	GPIO35
SD_CS_PIN	Chip select for SD card	GPIO16
SD_ENABLE_PIN	Enables power to SD card	GPIO4
SD_SPI	SPI bus number for SD card	SPI2
Miscellaneous		
DEBUG_LED_PIN	Onboard LED for debugging/status	GPIO15

4 System Processes

The following section describes the systems processes during a single cycle.

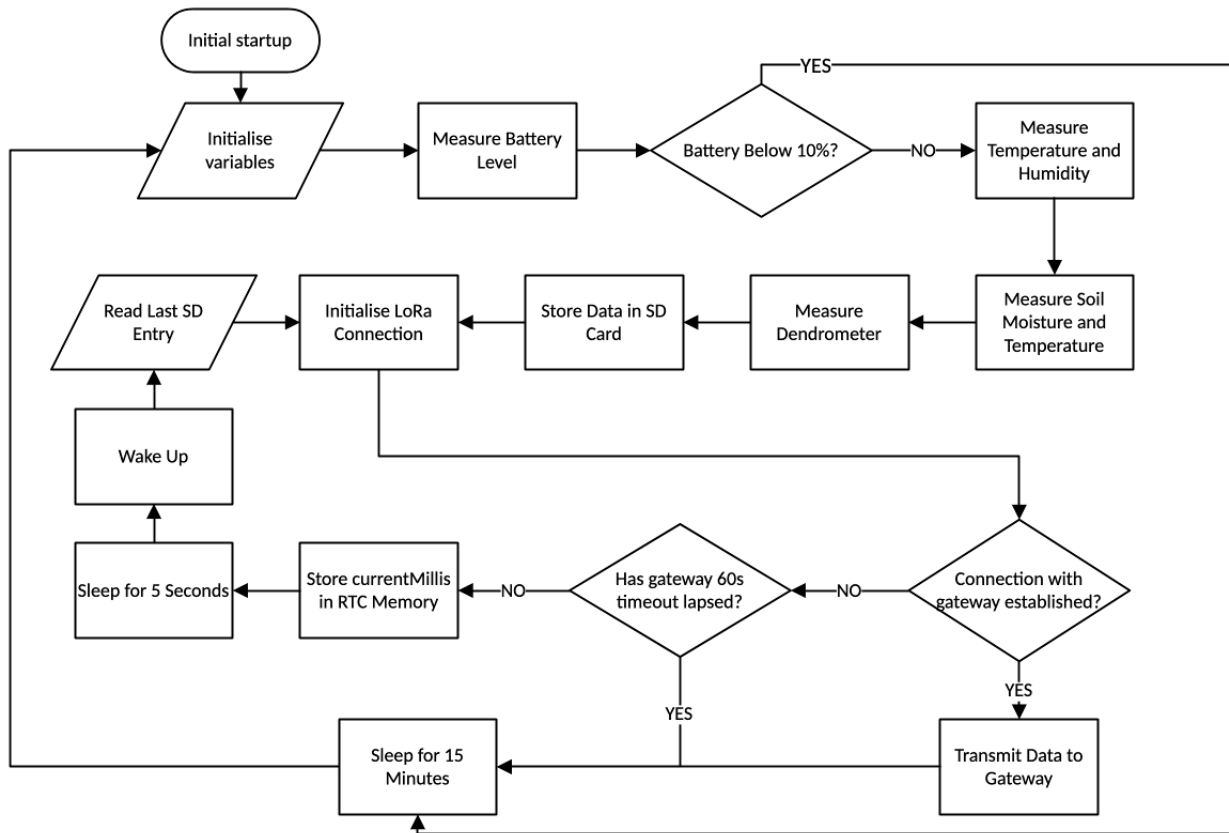


Figure 2: WiLo system high-level flow diagram

4.1 Initial Startup

4.1.1 Set CPU frequency to 40 MHz

This slows down the CPU to reduce active-mode current draw.

4.1.2 Activate WiFi capabilities for OTA (Over-the-Air) update

OTA update allows firmware to be loaded onto the ESP32 board wirelessly, using a Wi-Fi connection, without needing a physical connection to a computer. For more information on updating device firmware, see Section 7. The WiFi window stays open for 60 seconds in attempt to allow a WiFi capable device to connect, after which WiFi capabilities of the WiLo is disabled, until the device is reset.

4.1.3 Initialise SD Card

The device attempts to initialise the SD card module. If SD card initialisation is unsuccessful, the debug LED on the Lolin ESP32-S2 mini will flash for five seconds, then go into deep sleep for ten seconds.

4.1.4 Read/Write files from/to SD card

The system checks if a file called `Measurements.csv` is on the SD card. If not, the file is created. See Section 6 for interpretation of the files on the SD card. At this point the LoRa parameters are read from the file called `parameters.txt`. If no such file exists, or the system fails to read the content, the system goes into an infinite loop, where the debug LED stays ON for 3 seconds, and OFF for three seconds.

4.1.5 Check for dendrometer

The system attempts to connect to the digitised dendrometer. If no dendrometer is connected, the system simply ignores dendrometer measurements.

4.1.6 Setup Complete

A successful setup results in the debug LED staying on for around 2 seconds.

4.2 Measurements

All measurements are taken simultaneously instead of sequentially, in order to reduce system active time. For each sensor, a predefined number of samples (S) is taken. After sampling is done, the samples are ordered from smallest to largest, and the trimmed mean of a predefined number (N) of centremost values of the array are averaged, dropping significant outliers from the calculation.

4.2.1 Battery

The battery voltage is detected. 3.7 V Li-Po batteries have an output voltage ranging from 4.2 V at full charge (100 %) to around 3 V at depletion (0 %). S = 20, N = 5.

4.2.2 Air Temperature and Humidity (DHT22)

The DHT22 measurement is started. Note that in the case of a faulty sensor, different value codes are given to the DHT22 measurements (see Section ??). S = 10, N = 5.

4.2.3 Soil Temperature (DS18B20)

Similarly, the measurements of the DS18B20 sensor are taken. S = 5, N = 3.

4.2.4 Soil Moisture (Capacitive Soil Moisture V1.2) / Apogee Pyronometer

The Soil moisture probe as well as the Pyranometer are analogue devices. As there is no way to separate the two signals, only one of either sensors may be connected at a time. The output gives a raw voltage measurement, that must be converted to readable data. S = 10, N = 5.

4.2.5 Measure Dendrometer

Dendrometer measurements are taken. If no dendro is connected, the system simply returns 0 as the dendrometer measurement. S = 10, N = 5.

4.2.6 Measure Sapflow

The Sapflow sensor has the longest measurement time of all connected sensors. The upstream temperature (T1) and downstream temperature (T2) are sampled every second for 10 seconds, while the heatpulse is off. Then, the heating element is turned on, heating the stem, and T1 and T2 are sampled every second for 20 seconds. Finally, the heating element is turned off and T1 and T2 are sampled for an additional 10 seconds. The values of T1 and T2 are averaged for each state, and the Heat Pulse Velocity (HPV) estimate is calculated based on the following Marshall equation equation:

$$\text{HPV} = \frac{k}{x} \cdot \ln \left(\frac{\Delta T_2}{\Delta T_1} \right) \cdot 3600 \quad (1)$$

Where:

- HPV : Heat Pulse Velocity (mm/h), the velocity at which a thermal pulse moves through the medium.
- k : Thermal diffusivity constant (mm²), representing the effective cross-sectional area involved in heat conduction.
- x : Radial distance between the heater and the temperature sensors (mm).
- $\Delta T_1 = T_{1,\text{after}} - T_{1,\text{before}}$: Average temperature increase before and after the heat pulse at sensor 1 (upstream).
- $\Delta T_2 = T_{2,\text{after}} - T_{2,\text{before}}$: Average temperature increase before and after the heat pulse at sensor 2 (downstream).
- 3600 : Conversion factor from seconds to hours.

4.2.7 SDI-12

The WiLo checks to see if an SDI-12 device is connected. Note that the firmware only supports the Sentek SDI-12 Series III 90 cm and 60 cm drill and drop probes, and the Campbell Scientific CS655 Probe. Each SDI-12 device needs its address to be configured *before* connecting to the WiLo. Refer to Section 5 for installation and configuration considerations. If a SDI-12 device is connected, a file will be created, depending on the SDI-12 device connected. File names are as follows:

- **Sentek SDI-12 Series III 90 cm:** `SDI_12_90cm_Measurements.csv`
- **Sentek SDI-12 Series III 60 cm:** `SDI_12_60cm_Measurements.csv`
- **Campbell Scientific CS655:** `SDI_12_CS655_Measurements.csv`

if there is no SDI-12 device detected, the WiLo will simply omit the measurements from the payload.

4.3 Write Data to SD Card

Standard WiLo measurements are written to `Measurements.csv`, SDI-12 Measurements (if the device is connected) is written to the corresponding SDI-12 `SDI12_X_Measurements.csv` file.

4.4 LoRa Setup

Once all measurements are done, the WiLo creates a LoRa payload. The payload is passed on to the RFM95W LoRa module, which attempts to connect to a nearby LoRa gateway using the keys stored in the `parameters.txt` file. If a successful connection is made, the LoRa payload is transmitted to the gateway and the WiLo goes into deep sleep. In the case of no connection, the device goes into a light sleep for 5 seconds, wakes up, attempts to transmit the payload again. This is done until a timer reaches 60 seconds, after which the device goes to deep sleep.

4.5 Wake-up Procedures

After an active cycle of measuring, storing data in the SD card and transmitting data over LoRa, the WiLo wakes up after some time. In the current firmware, the WiLo keeps track of the active time and adjusts the deep sleep time accordingly. The total cycle time is 15 minutes, in order to keep measurements approximately 15 minutes apart. Total cycle time can be expressed as:

$$\text{Cycle Time} = \text{Active Time} + \text{Deep Sleep Time} = 15 \text{ minutes} \quad (2)$$

For each consecutive wake-up, the device runs through processes 4.1 – 4.4.

5 SDI-12 Installation and Configuration

Before installation, the SDI-12 device must be programmed with a dedicated device address. The user must also adjust the power supply voltage to the SDI-12 device manually.

5.1 SDI-12 Memory Addressing

Table 3 shows the SDI-12 devices supported by the current WiLo version and the device addresses that must be written to the SDI-12 devices before installation. This step is critical in ensuring flawless measurements from these devices. Changing the device address can be done using an Arduino, or a Campbell scientific data logger.

- **Arduino:** The [Arduino-SDI-12 library](#) provides ample information, source code and examples on interfacing SDI-12 sensors to Arduino MCUs. **Warning: Arduinos can provide 5 V to the Sentek Series III drill and drop probes without issue, but other SDI-12 devices (such as the CS655) require 6—12 V power. Therefore, an external power supply is strongly recommended when writing memory addresses of SDI-12 devices that do not accept 5 V power.**
- **Campbell Scientific dataloggers:** To assign a new memory address to an SDI-12 sensor using a Campbell Scientific data logger, you can use the logger's terminal emulator to enter *transparent mode*. In this mode, send the command “aAb!”, where a is the current sensor address and b is the desired new address. For example, to change the address from 0 to B, input “0AB!”. The sensor should respond with the new address, confirming the change. This method is supported by several Campbell Scientific loggers, such as the CR1000X, CR6, CR300, and others. For more information and detailed instructions, refer to the following resources:
 - [Changing an SDI-12 Address](#)
 - [SDI-12 Transparent Mode](#)

SDI-12 Device	Required Address
Sentek SDI-12 Series III 90 cm	S
Sentek SDI-12 Series III 60 cm	J
Campbell Scientific CS655	6

Table 3: Current SDI-12 support and required memory addresses

- [Change Address Command \(aAb!\)](#)

5.2 SDI-12 Voltage Configuration

Before an SDI-12 sensor is connected to a WiLo, the voltage supply to the sensor must be configured manually.

You will require:

- A male to male dupont jumper cable.
- A digital multimeter for voltage measurements.
- A small flathead screwdriver for adjusting the potentiometer.

The steps are as follows:

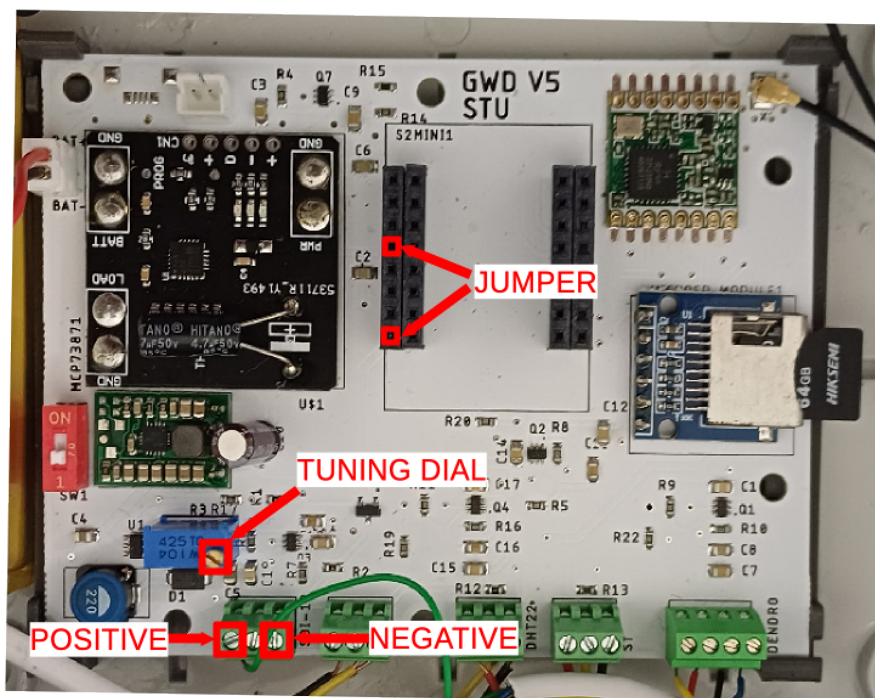


Figure 3: SDI-12 voltage configuration notes. JUMPER: The header pins to which the jumper cable must be connected to. TUNING DIAL: The tuning dial of the potentiometer (CW: +V, CCW: -V). POSITIVE: The positive terminal of the SDI-12 terminal (+ Multimeter probe). NEGATIVE: The negative terminal of the SDI-12 terminal (- Multimeter probe).

1. Turn off the WiLo by putting the red power switch in the off position.
2. Remove the Lolin ESP32-S2 Mini from its headers on the WiLo.

3. Connect the jumper wire to two pins on the headers. In the leftmost row of the headers, connect the wire to the fourth pin from the top, and the last pin from the top (Fig. 3). This supplies a 3.3 V signal to the power circuitry, that turns on the boost regulator responsible for powering the SDI-12 sensor.
4. Turn on the WiLo by putting the red power switch in the on position.
5. Put the multimeter on the voltage reading function, and place the positive probe (red) on pin 1 and the negative probe on pin 3 (See Fig. 1 and Fig. 3). You should see a voltage level display on the multimeter.
6. Using the flathead screwdriver, turn the dial of the blue potentiometer clockwise to increase the boost voltage, and anticlockwise to decrease the boost voltage. The voltage required is specific to the SDI-12 probe used:
 - **Sentek Series III Probes:** 5 V.
 - **CS655:** 6 V.
7. Once the desired voltage level is reached, turn off the WiLo and resituate the Lolin ESP32-S2 mini in the headers.
8. Connect the SDI-12 device wires to the terminals according to the pin reference of Section 2.
9. Turn the WiLo on again and wait for approximately 3 minutes to see if the payload has been transmitted to TTN, or alternately perform a serial debug to see whether values were measured correctly.

6 SD Card

6.1 Formatting SD card

Before the WiLo can be deployed, the SD card must be formatted and the `LoRa parameters.txt` file must be copied to the SD card. Formatting is done with these steps:

1. Place the SD card in a micro-SD adapter and connect it to a PC.
2. format the SD card to FAT32. This is of utmost importance, as no other SD format is supported. Windows PCs do not allow SD card sizes greater than 32 GB to be formatted to FAT32. There is a workaround for this; using a third party tool. for a full guide, see <https://www.easeus.com/partition-master/format-64gb-sd-card-to-fat32.html>.
3. Copy the LoRa parameter file to the SD card. Backups can be found here: [GitHub Repository](#)
4. Insert the SD card in the SD card slot of the WiLo.

6.2 SD Card Measurement Files

On the SD card, there will be up to measurement files. The first, which is the default of all WiLos, is `Measurements.csv`.

Table 4: Descriptions of columns in the `Measurements.csv` file

Name	Description	Unit
Dendrometer	Digitised dendrometer measurement	μm
Air Temperature	DHT22 air temperature at the Canopy	$^{\circ}\text{C}$
Air Humidity	DHT22 relative humidity at the canopy	%
SM or Pyro	Voltage output of the Capacitive Soil Moisture sensor or Apogee SP-110	mV
Battery	Battery percentage	%
SF T1 Before	Average temperature of upstream sapflow temperature sensor (T1) before heat pulse	$^{\circ}\text{C}$
SF T2 Before	Average temperature of downstream sapflow temperature sensor (T2) before heat pulse	$^{\circ}\text{C}$
SF T1 During	Average temperature of upstream sapflow temperature sensor (T1) during heat pulse	$^{\circ}\text{C}$
SF T2 During	Average temperature of downstream sapflow temperature sensor (T2) During heat pulse	$^{\circ}\text{C}$
SF T1 After	Average temperature of upstream sapflow temperature sensor (T1) after heat pulse	$^{\circ}\text{C}$
SF T2 After	Average temperature of downstream sapflow temperature sensor (T2) after heat pulse	$^{\circ}\text{C}$
HPV	Estimated HPV from Marshall formula (Eq.1)	m/h (metres per hour)
Bootcount	keeps track of the number of cycles. The value needs to be multiplied by 100 to get the actual boot count	number of cycles

7 Firmware Upgrade

Firmware can be upgraded through two ways: Through a physical connection with a PC, or Over the Air (OTA) using a mobile device or PC. This section provides details on how to accomplish firmware upgrade with either method.

7.1 Configuring PlatformIO and Source Files in VS Code

1. Install the [PlatformIO IDE](#) within [Visual Studio Code](#). For installation instructions, refer to the [PlatformIO IDE for VSCode Documentation](#).
2. Download the WiLo source files from the [WiLo GitHub Repository](#).
3. Unzip the `WiLo-main.zip` file to a location on your PC, and take note of the directory path.
4. Open Visual Studio Code and click on the PlatformIO icon in the left toolbar.
5. In the Quick Access section, click on Open.
6. On the PlatformIO welcome screen, click Open Project. This opens a file navigation window.
7. Navigate to the `WiLo-main/WiLo Code` directory (where the ZIP file was extracted). Inside the `WiLo Code` folder, ensure there is a `platformio.ini` file. Select the folder and click Open "WiLo Code".
8. A popup will prompt you to trust the authors of the files. Select Yes. PlatformIO will now configure the project, which may take a few minutes.
9. At this point the project files are ready to be flashed to the WiLo.

7.2 Physical Connection Firmware Upgrade

1. After configuration of Section 7.1 is completed, click the PlatformIO icon again on the left toolbar.
2. Connect the ESP32-S2 Mini board to your PC using either a USB-C to USB-A or USB-C to USB-C cable.
3. To enter download mode on the ESP32-S2:
 - Hold down the $\emptyset$ button (labelled EN or $\emptyset$).
 - While holding it, press the RST (Reset) button once.
 - Release the RST button.
 - Finally, release the $\emptyset$ button.

This sequence is: HOLD EN $\rightarrow$ PRESS RST $\rightarrow$ RELEASE RST $\rightarrow$ RELEASE EN.

4. If done correctly, the device will be ready to flash. On the bottom bar of VS Code, click the plug icon labeled Auto.
5. A dropdown will appear at the top of VS Code. Select the appropriate COM port. It may be labeled similarly to COM6 : Lolin ESP32-S2 mini Dev Module (your port number may differ).
6. In the left pane under Project Tasks, click Upload. This will compile the project and flash it to the microcontroller. The Terminal will show flash progress as a percentage.
7. Note: the terminal may end with an error message — this simply indicates that the ESP32-S2 Mini was placed into download mode manually. To run the program:
 - Press the RST button again, or
 - Power cycle the board by unplugging the USB cable and reconnecting the USB cable.

7.3 Wireless Connection (OTA) Firmware Upgrade

A WiLo can be flashed wirelessly. This requires the `firmware.bin` file to be retrieved beforehand. Note that this is only intended for future WiLo source code changes, as some WiLos might have an older version of the firmware that does not support OTA. If a WiLo was physically flashed with the latest firmware before field deployment, OTA is possible.

7.3.1 Retrieving `firmware.bin` File

- Ensure that PlatformIO is configured as per Section 7.1.
- Click on the PlatformIO icon on the left tab.
- Under Project Tasks click Build.
- Once build is completed (it will say SUCCESS in the terminal), click Build once more. This ensures that the latest version of the firmware is copied.
- In the terminal, you should see a line saying: “Copied firmware.bin to: /Users/.../WiLo Code/firmware/firmware.bin”.

7.3.2 Setting up mobile device/PC for OTA

The steps described below are crucial for allowing the WiLo to connect wirelessly. OTA can be done with either a PC or a smartphone.

1. On a PC or smartphone, add a new WiFi network with the following credentials.
 - **SSID:** CSE
 - **PASSWORD:** 12345678
2. Copy the `firmware.bin` to the device doing the firmware upgrade.
3. Turn off the WiLo, and turn it on again. This will reset the WiLo, putting it in a WiFi access point state for 60 seconds until a device connects to it. If no device connects within the sixty seconds, it will simply restart, disabling WiFi to conserve power, and continue normal operation.
4. Upon resetting the WiLo, attempt to connect to the WiLo hotspot. Once connection is established, the WiLo will turn on the blue debug LED, to indicate that a device is connected.
5. On the connected device, go to the network settings, and determine the IP-Address of the WiLo WiFi network.
6. Copy the IP address to a browser on the connected device. You will be greeted with the firmware update webserver of the WiLo.
7. In the example below, a Macbook was used. The IP address was found to be 192.168.4.1. Once entered in the browser the following appears:

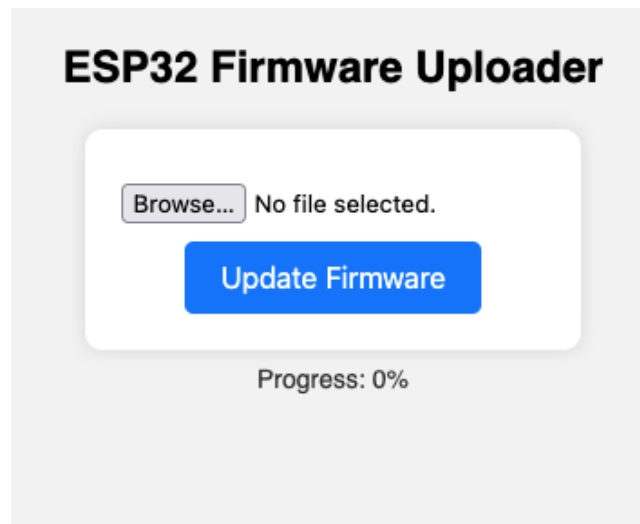


Figure 4: Firmware updater webserver of the WiLo. Click on “Browse”, select the `firmware.bin` file, and click “Update Firmware”.

8. Click on “Browse”, select the `firmware.bin` file, and click on “Update Firmware”. The progress bar will show the progress completion percentage of the Update.
9. When the update is completed, the WiLo will restart itself and a new WiFi window will be opened. You can simply disconnect from the WiLo, or turn off the WiFi of the connected device. The WiLo will restart one final time, and the new firmware will be in place for normal operation to continue.

8 Debugging

This section will describe some key hardware debugging tips.

8.1 Hardware Debugging

8.1.1 Battery

If the battery level is at around 10%, the WiLo immediately goes into deep sleep for some time (15 min). This reduces power consumption, in the attempt of allowing the battery to recharge via solar before sensor measurements are taken, or a payload is transmitted. However, the batteries may struggle to recharge sufficiently long term when at this critical level, especially during low-sunlight conditions. Therefore, it is the technician's responsibility to swap out the battery for a fully charged battery in this instance.

Table 5: Battery Voltage vs State of Charge (10% increments)

SoC (%)	Battery Voltage (V)
100	4.200
90	3.936
80	3.847
70	3.803
60	3.769
50	3.737
40	3.698
30	3.685
20	3.635
10	3.350
0	3.000

To measure the battery voltage, a multimeter is required. Place the positive and negative probes of the multimeter on the positive and negative terminals of the MCP73871 battery charging module (Fig. 5). Correspond the voltage read by the multimeter to a SoC% as shown in Table 5.



Figure 5: Battery debug measurement terminals.

8.1.2 SD Card Errors

In some cases the SD card will not be recognised by the WiLo. This will result in a loop where the debug LED will flash. This is due to one of two possibilities:

- The SD card is not in the correct format. Refer to Section 6 for information on SD card formatting.
- The SD card module connections have issues. This will require some technical skill with a multimeter and a soldering iron. To fix this issue, refer to the PCB layout and schematic in Appendix B for connections, and test continuity with a multimeter. If a poor connection is found, or a visual inspection verifies a poor solder connection do the following:
 1. Turn the WiLo off, disconnect all wires in screw terminals, and remove the battery.
 2. Remove the WiLo from the enclosure.
 3. Use a heatgun, and heat up the area on the *underside* of the PCB around the connections of the SD card module. This will allow the solder to flow more easily, especially around the ground pin.
 4. Use a soldering iron to heat up the solder pad around a pin. Try to make good contact with both the pin and the solder pad.
 5. Feed a tiny amount of solder to the connection and hold the soldering iron there until the solder “latches” onto the solder pad and the pin.
 6. If all the connections seem well connected, reconnect all the sensors and the battery and turn on the WiLo to see if the SD card is connected. Serial debugging is advised.

8.2 Sensor Errors

The following table will describe the error codes used to determine whether sensors might be faulty/not properly connected. The error codes are stored in the `Measurements.csv` file in the expected measurement column of the faulty sensor.

Table 6: Sensor Error Codes and Troubleshooting

Sensor	Error Code	What to do
DHT22	1111.11 (Temp/Hum out of bounds: $T < -40^{\circ}\text{C}$ or $T > 80^{\circ}\text{C}$, $H < 0\%$ or $H > 100\%$)	Replace sensor
	9999.99 (Unresponsive/Not connected: $T = \text{NaN}$ or $H = \text{NaN}$)	Check connections/Replace
SM	Voltage reading stays at 2600mV	Replace sensor
DS18B20 Soil Temp	1111.11 (Temp out of bounds: $T < -40^{\circ}\text{C}$ or $T > 80^{\circ}\text{C}$)	Replace sensor
	9999.99 (Unresponsive/Not connected: $T = \text{NaN}$)	Replace
Sapflow Sensor	1111.11 (Not connected/Faulty)	Check connections/Replace
Dendrometer	No error codes in current firmware	N/A
SDI-12	No error codes in current firmware	N/A

Appendices

Appendix A: TTN Payload Formatter

The WiLo encodes the payload of measurements into three bytes per measurement for simplicity and handling floating point numbers. The payload can be in one of four cases:

- **Default case:** The default case applicable to all WiLos. This case accounts for the first 42 bytes of the payload (14 values $\times$ 3 bytes = 42 bytes).
- **CS655 case:** a Campbell Scientific CS655 sensor is added to a default WiLo. The payload size increases to 51 bytes to account for the measurements taken by the CS655 (42 bytes + 3 values $\times$ 3 bytes = 51 bytes).
- **Sentek Series III 60 cm:** A Sentek Series III 60 cm drill and drop probe is added. This probe measures 6 temperature and 6 soil moisture at 10 cm increments along the length of the probe (into the depth of the soil), thereby increasing the default length of the payload by 36 bytes, to a total of 78 bytes (42 bytes + 12 values $\times$ 3 bytes = 78 bytes).
- **Sentek Series III 90 cm:** A Sentek Series III 90 cm drill and drop probe is added to a default WiLo. This probe measures 9 temperature and 9 soil moisture at 10 cm increments along the length of the probe (into the depth of the soil), thereby increasing the default length of the payload by 54 bytes, to a total of 96 bytes (42 bytes + 18 values $\times$ 3 bytes = 96 bytes).

NOTE: Only a single SDI-12 sensor can be connected to a WiLo at any given time. There won't be any cases other than the ones mentioned above. Not all WiLos will have a SDI-12 sensor connected, and will fall under the default case.

Payload formatter code

The following JavaScript function decodes a byte array ('b') received from a WiLo over LoRa. It extracts sensor readings such as temperature, humidity, and soil moisture, and organizes them into a structured 'decoded' object. The decoding logic adjusts based on the length of the payload. Descriptions of fields can be found in Table 7.

```
function Decoder(b, port) {
  var values = [];
  var j = 0;
  var i = 0;
  var payloadLength = b.length; // Determine actual payload size

  // Ensure we don't exceed the bounds while processing 3-byte groups
  while (i + 2 < payloadLength) {
    values[j] = b[i] * 100 + b[i + 1] + b[i + 2] / 100.0;
    j++;
    i += 3;
  }

  // Base object
  var decoded = {
    payloadLength: payloadLength,
    Dendro: values[0]*100, // Dendrometer (um)
    AirTemp: values[1], // Air Temperature (C)
    AirHumidity: values[2], // Air Humidity (%RH)
    SoilTemp: values[3], // Soil Temperature (C)
    SoilMoistureOrPyranometer: values[4], // Soil Moisture (SMV) or Pyranometer measurement (V)
    BatteryLevel: values[5], // Battery level (%)
    T1Before: values[6], // T1 (upstream) Avg Temp BEFORE heat pulse (C)
    T2Before: values[7], // T2 (downstream) Avg Temp BEFORE heat pulse (C)
    T1During: values[8], // T1 (upstream) Avg Temp DURING heat pulse (C)
  }
}
```

```

    T2During: values[9], // T2 (downstream) Avg Temp DURING heat pulse (C)
    T1After: values[10], // T1 (upstream) Avg Temp AFTER heat pulse (C)
    T2After: values[11], // T2 (downstream) Avg Temp AFTER heat pulse (C)
    HPVMarshall: values[12], // HPV using Marshall formula
    BootCount: values[13] // Boot count
};

// Add extra fields if payload length is greater than 42
if (payloadLength > 42) { // if greater than default
    if (payloadLength == 51) { // CS655 payload
        Object.assign(decoded, {
            CS655SoilMoisture1 : values[14],
            CS655SoilMoisture2 : values[15],
            CS655Temperature : values[16],
        });
    } else {
        Object.assign(decoded, { // Add extra fields for Sentek Series III
            SDISoilMoisture1: values[14],
            SDISoilMoisture2: values[15],
            SDISoilMoisture3: values[16],
            SDISoilMoisture4: values[17],
            SDISoilMoisture5: values[18],
            SDISoilMoisture6: values[19]
        });

        // Add even more fields if payload length is greater than 78
        if (payloadLength > 78) { // Sentek Series III 90cm
            Object.assign(decoded, {
                SDISoilMoisture7: values[20],
                SDISoilMoisture8: values[21],
                SDISoilMoisture9: values[22],
                SDITemp1: values[23],
                SDITemp2: values[24],
                SDITemp3: values[25],
                SDITemp4: values[26],
                SDITemp5: values[27],
                SDITemp6: values[28],
                SDITemp7: values[29],
                SDITemp8: values[30],
                SDITemp9: values[31],
            });
        } else { // Sentek Series III 60cm
            Object.assign(decoded, {
                SDITemp1: values[20],
                SDITemp2: values[21],
                SDITemp3: values[22],
                SDITemp4: values[23],
                SDITemp5: values[24],
                SDITemp6: values[25]
            });
        }
    }
}

return decoded;
}

```

Field	Description
payloadLength	Number of bytes in the received payload
Dendro	Dendrometer measurement (μm)
AirTemp	Air temperature ($^{\circ}\text{C}$)
AirHumidity	Air humidity (% RH)
SoilTemp	Soil temperature ($^{\circ}\text{C}$)
SoilMoistureOrPyranometer	Soil moisture (SMV) or pyranometer reading (analogue, V)
BatteryLevel	Battery remaining capacity (%)
T1Before, T2Before	Avg sapflow temperatures before heat pulse (T1: upstream, T2: downstream) ($^{\circ}\text{C}$)
T1During, T2During	Avg sapflow temperatures during heat pulse (T1: upstream, T2: downstream) ($^{\circ}\text{C}$)
T1After, T2After	Avg sapflow temperatures after heat pulse (T1: upstream, T2: downstream) ($^{\circ}\text{C}$)
HPVMarshall	Heat pulse velocity using Marshall formula (m/h)
BootCount	Number of times device has booted.
Optional SDI-12 sensor fields (if present):	
CS655SoilMoisture1,2	Soil moisture readings from CS655 probe
CS655Temperature	Temperature from CS655 probe
SDISoilMoisture1-9	Soil moisture readings from SDI-12 sensor
SDITemp1-9	Corresponding temperature readings from SDI-12 sensor
These may come from a Sentek Drill & Drop probe (60 or 90 cm), or a CS655 sensor	

Table 7: Description of decoded payload fields

Appendix B: PCB Layout and Schematic

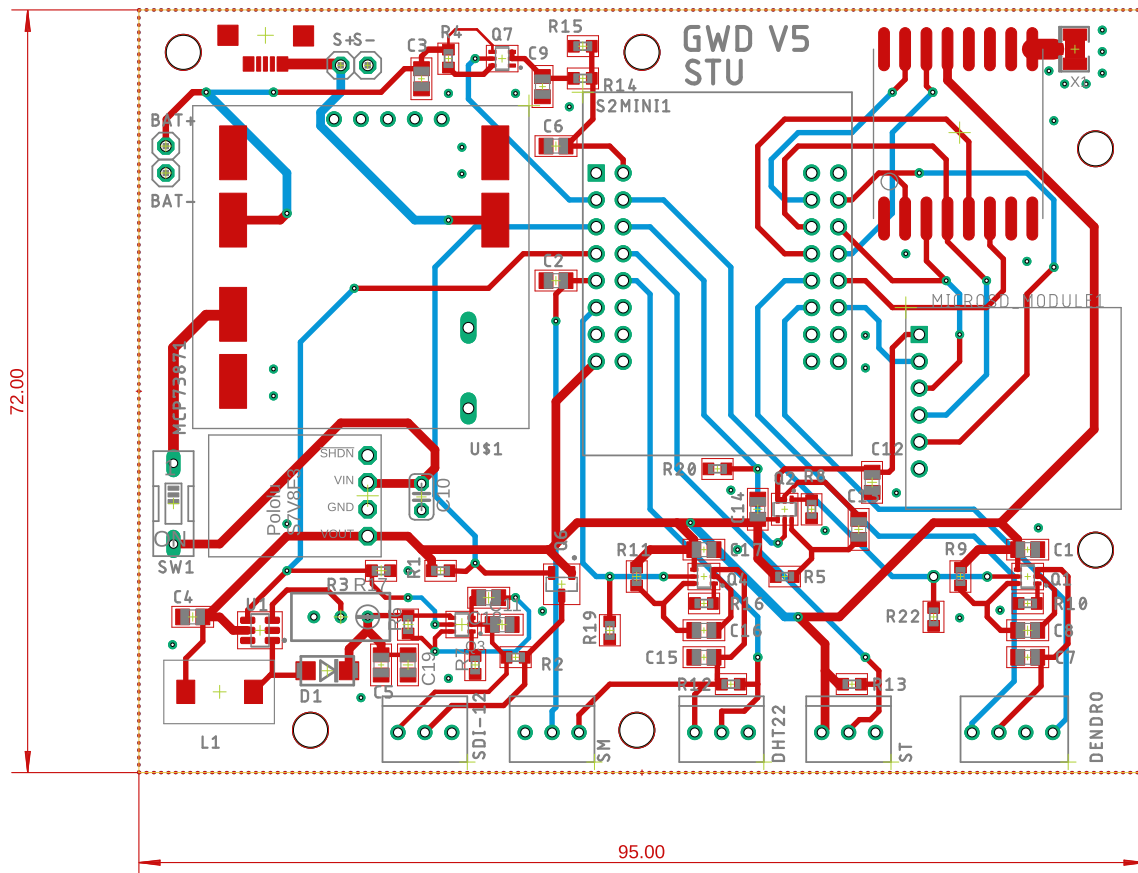


Figure 6: PCB Layout of the WiLo. Red: Top Layer. Blue: Bottom layer.

